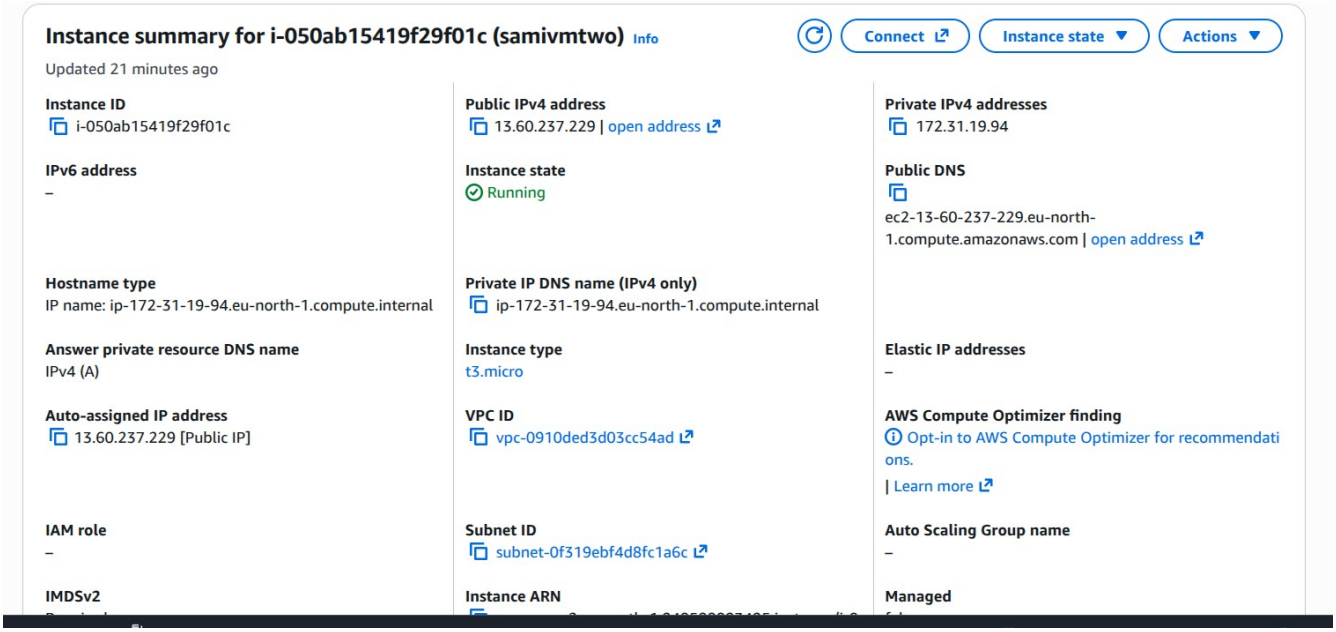
Website Deployment on AWS EC2 Using Nginx

This report documents the complete deployment of a static hostel accommodation website on an **Amazon EC2** Ubuntu instance using the **Nginx web server.** Every step is shown with a screenshot, and each screenshot is preceded by a short note explaining what it shows and why that step is needed.

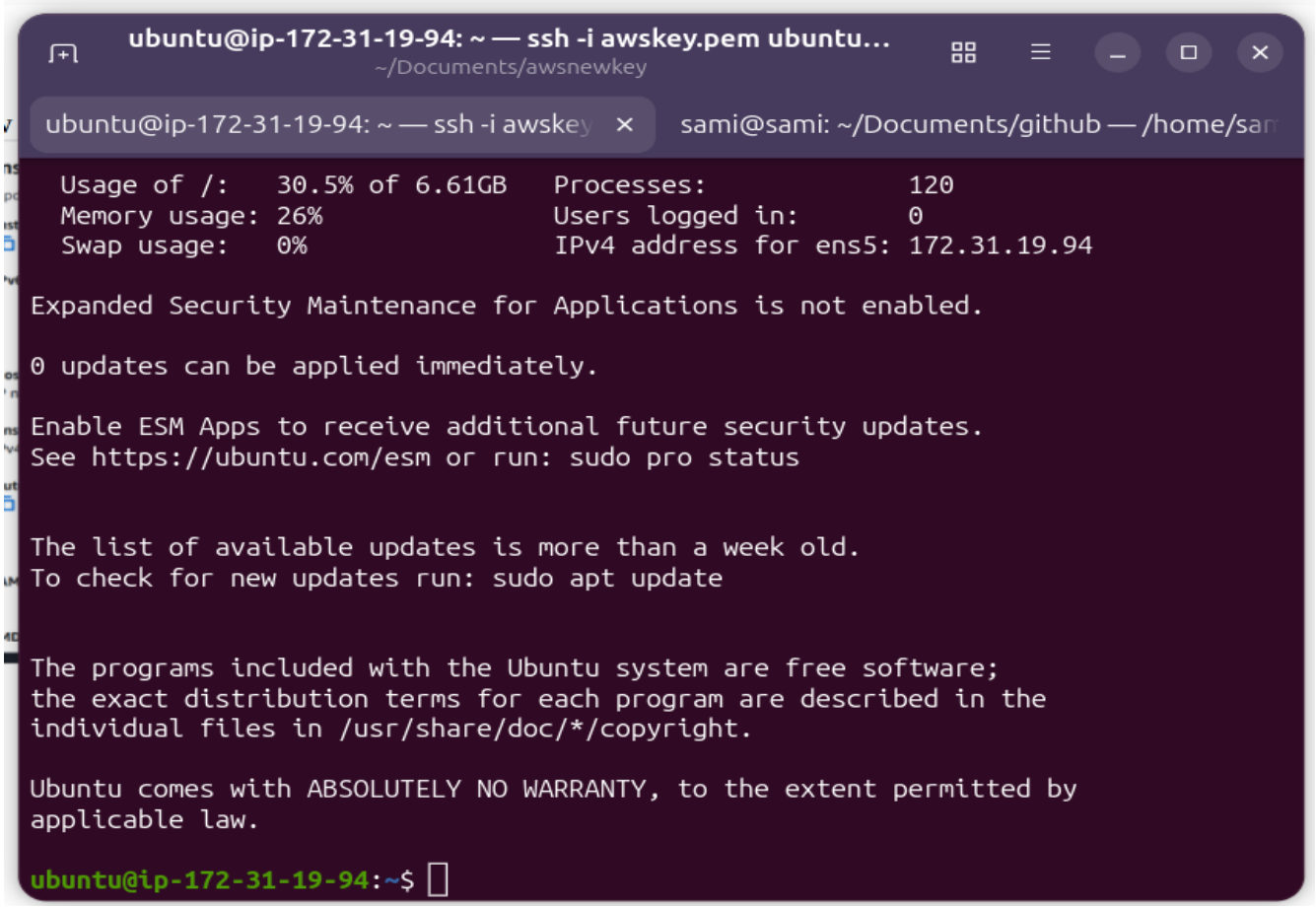
1. Launching the EC2 Instance

A new EC2 instance named `samivmtwo` has been created for completing the website deployment task. The instance summary confirms that it is in the **Running** state and gives the public IPv4 address **13.60.237.229**. This machine acts as the cloud server on which the website will finally be hosted.



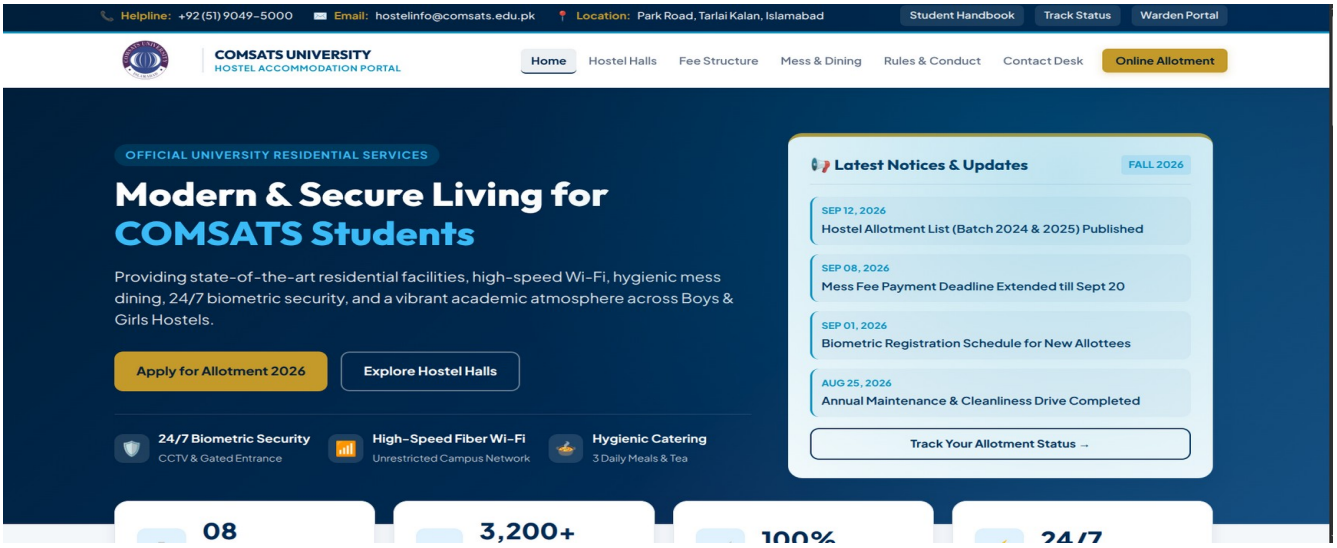
2. Connecting to the Cloud VM

Here we are connected to the cloud VM from the local terminal using SSH with the `awskey.pem` key file. The Ubuntu welcome banner confirms that the login was successful. From this point onwards every command is executed on the EC2 machine and not on the local system.



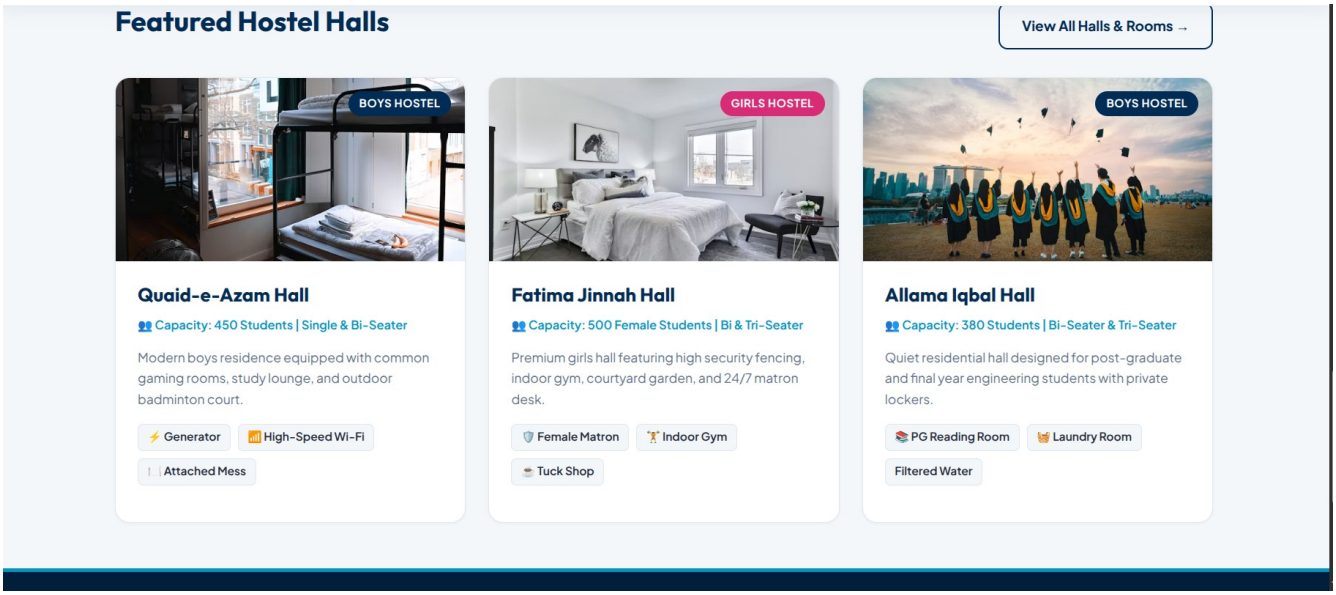
3. Website Frontend Developed and Pushed to GitHub

This is the **frontend** of the website that was developed locally and pushed to a GitHub repository. It is the COMSATS hostel accommodation portal, built with plain HTML and CSS. It is shown here to record what the site looks like before it is deployed on the server.

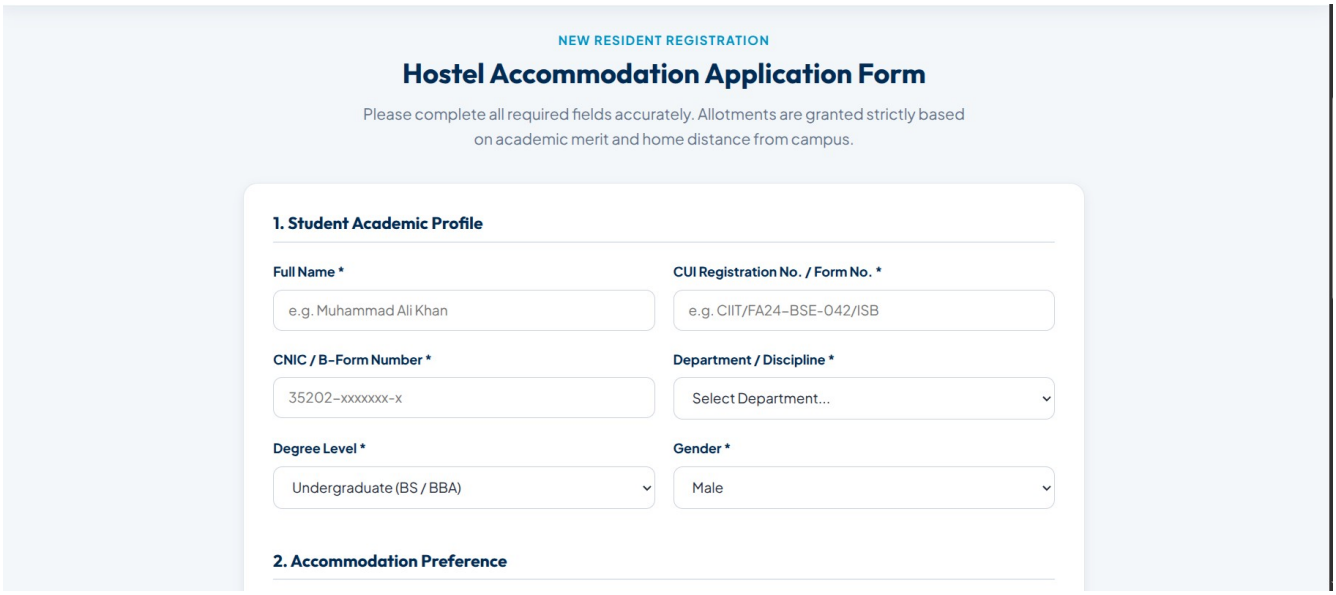


Second page screenshots:

A few more pages of the developed site are shown here. This section lists the featured hostel halls along with their capacity and facilities. It is included to show that the project contains several linked pages and not just a single index file.

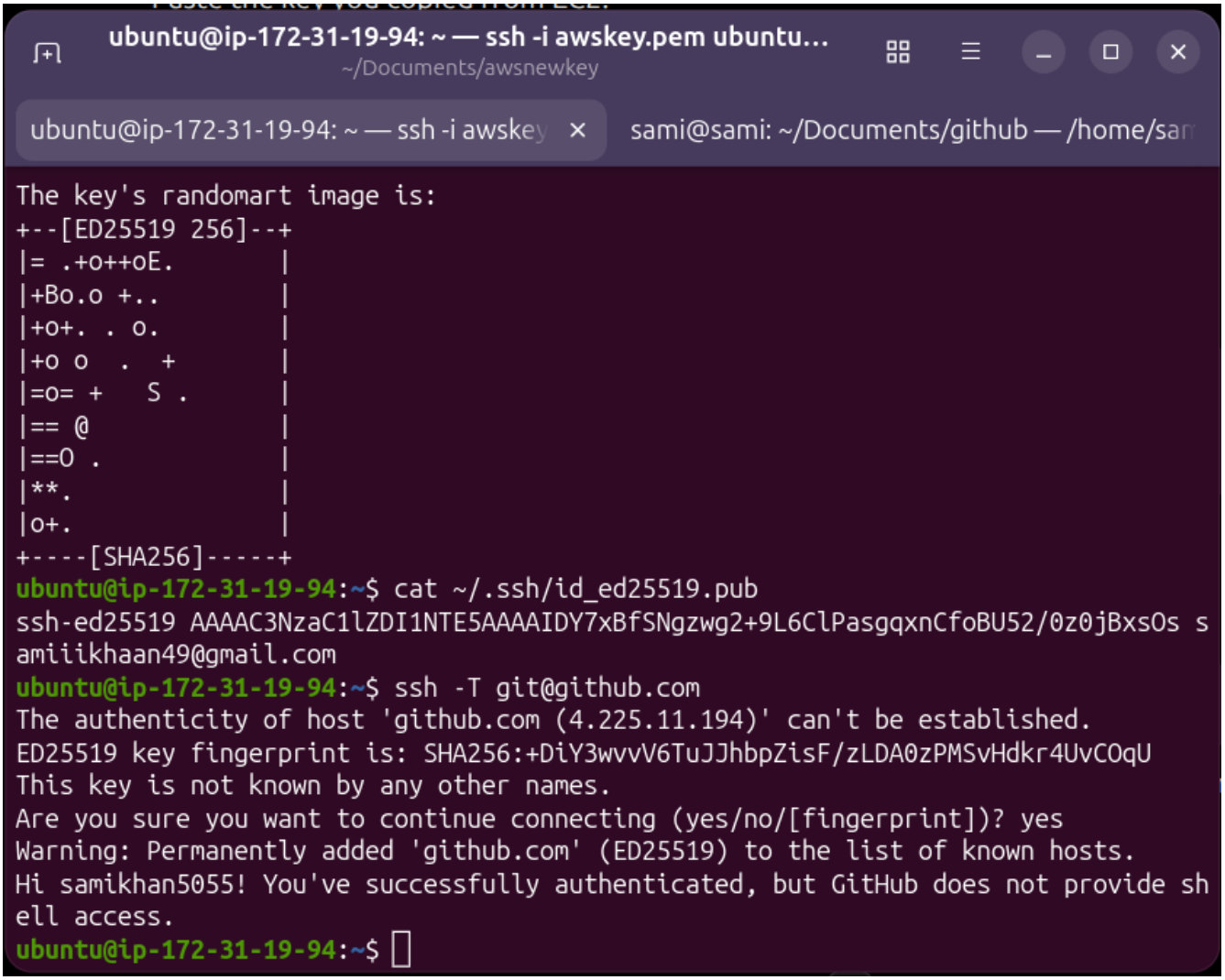


This is the application form frontend page for hostel allotment. It collects the student academic profile and the accommodation preference details. It is shown so that the same page can later be verified once the site is live on the server.

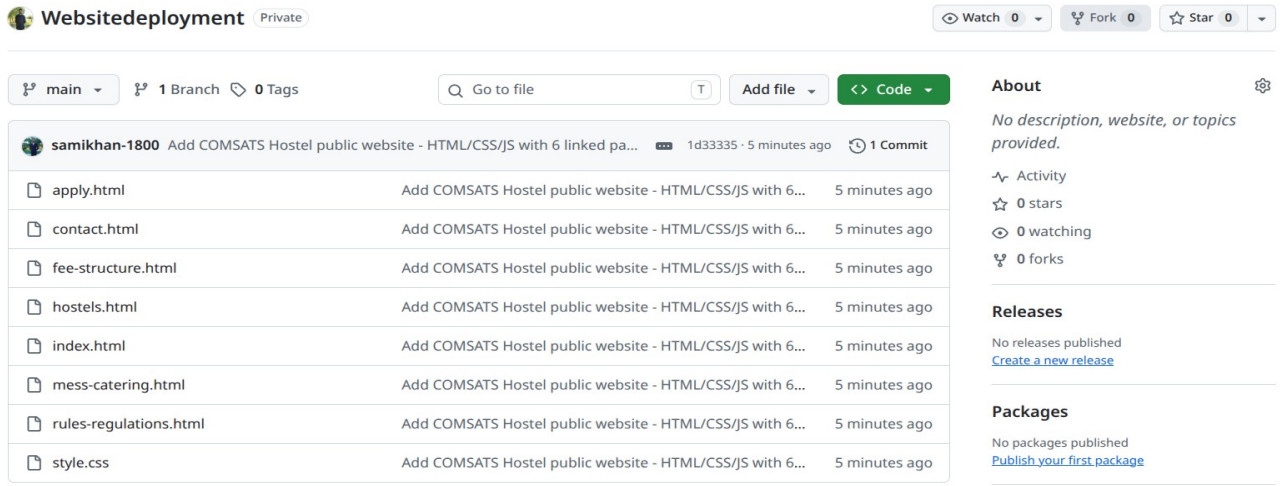


4. Connecting GitHub with the EC2 Instance

Now we have connected GitHub to the EC2 machine from the Ubuntu terminal. The public SSH key is printed with `cat ~/.ssh/id_ed25519.pub` and added to the GitHub account, then **`ssh -T git@github.com`** is run to test it. The success message confirms the server can now reach the repository without entering a password.

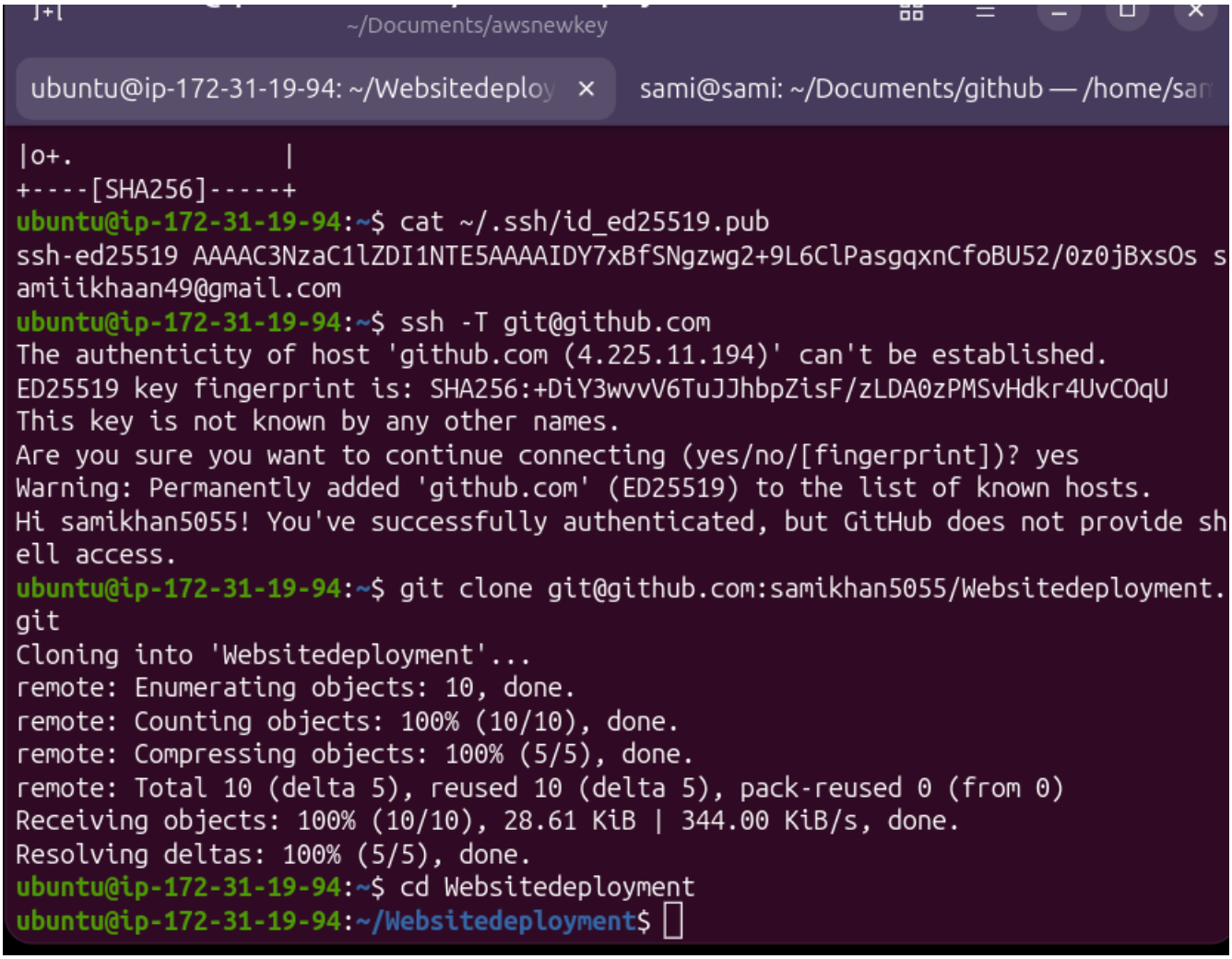


This is the newly created repository `Websitedeployment` containing all the files of the site that will be deployed. Pages such as `index.html`, `apply.html`, `contact.html` and `style.css` are visible in the listing. This is the repository that will be cloned onto the cloud VM.

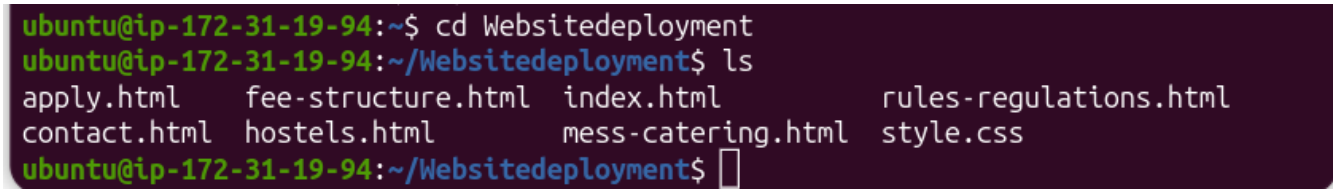


5. Cloning the Repository on the Server

The repository is being cloned on the cloud VM using **`git clone` over SSH**, and then we change into the `Websitedeployment` directory. Cloning is used so that exactly the same code that is on GitHub is placed on the server. This avoids uploading files manually and keeps the two copies in sync.



Now we are displaying all the files inside the repository that was cloned, using the **`ls`** command. The listing shows every HTML page along with the stylesheet. This check is done to make sure the whole project was downloaded correctly before copying it into the web server directory.



6. Installing and Configuring the Nginx Web Server

Following commands were run in the terminal for the setup of the web server.

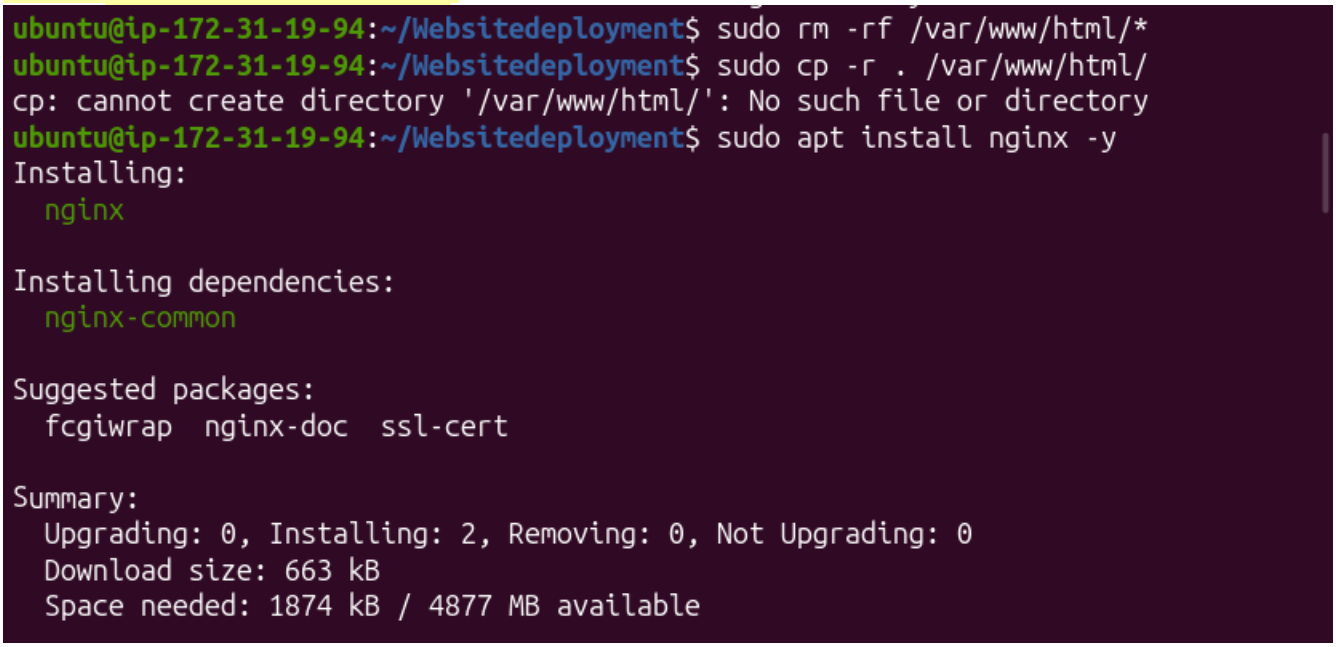
`sudo rm -rf /var/www/html/*`

This command removes the default files from Nginx's web directory. It clears the existing Nginx welcome page so that it can be replaced with our own website.

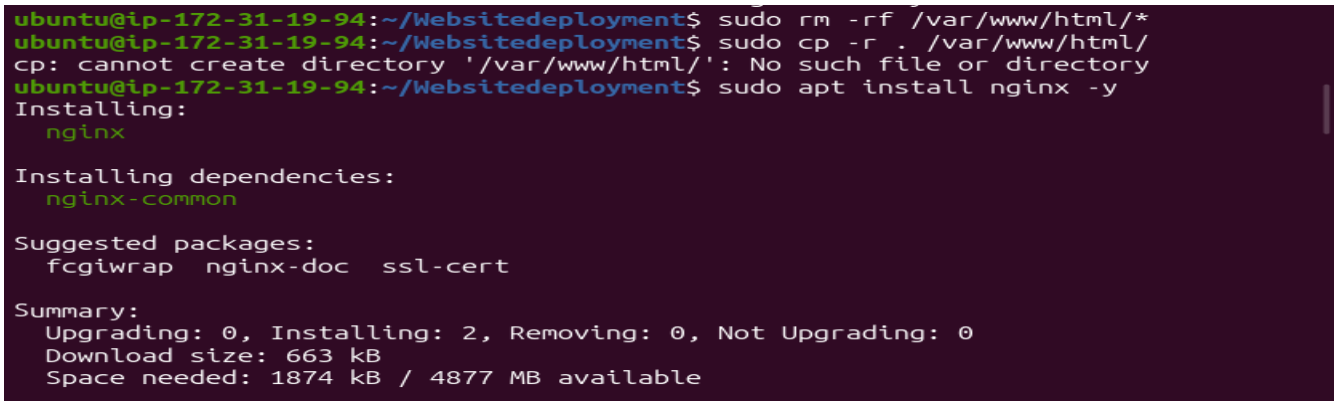
`sudo cp -r . /var/www/html/`

This command copies all files from the current `Websitedeployment` GitHub folder into Nginx's web directory. The HTML, CSS, JavaScript and other website files are therefore placed where Nginx can serve them.

This screenshot shows `sudo apt install nginx -y` being run, which installs the **Nginx web server** on the EC2 Ubuntu machine. Nginx is required because the EC2 instance has no web server of its own and cannot serve pages without one. The installation completed successfully with Nginx version `1.28.3-2ubuntu1.11`.



This installs the **Nginx web server** on the EC2 Ubuntu machine. The installation completed successfully with Nginx version `1.28.3-2ubuntu1.11`.



Here `sudo systemctl status nginx` is used to check the state of the web server service. The output shows the service as **active (running)** and enabled at boot. This confirms that Nginx is live and ready to serve files from `/var/www/html`.

```
No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-19-94:~/Websitedeployment$ sudo systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
     Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enab>
     Active: active (running) since Tue 2026-09-15 08:49:10 UTC; 25s ago
 Invocation: 4c951a02e2104de9922dae44b16cb3cb
       Docs: man:nginx(8)
    Process: 2024 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process>
    Process: 2026 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (cod>
   Main PID: 2055 (nginx)
      Tasks: 3 (limit: 627)
     Memory: 3.1M (peak: 7.3M)
        CPU: 43ms
     CGroup: /system.slice/nginx.service
             ├─2055 "nginx: master process /usr/sbin/nginx -g daemon on; master_p>
             ├─2058 "nginx: worker process"
             └─2059 "nginx: worker process"

Sep 15 08:49:10 ip-172-31-19-94 systemd[1]: Starting nginx.service - A high perfo>
Sep 15 08:49:10 ip-172-31-19-94 systemd[1]: Started nginx.service - A high perfor>
lines 1-18/18 (END)
```

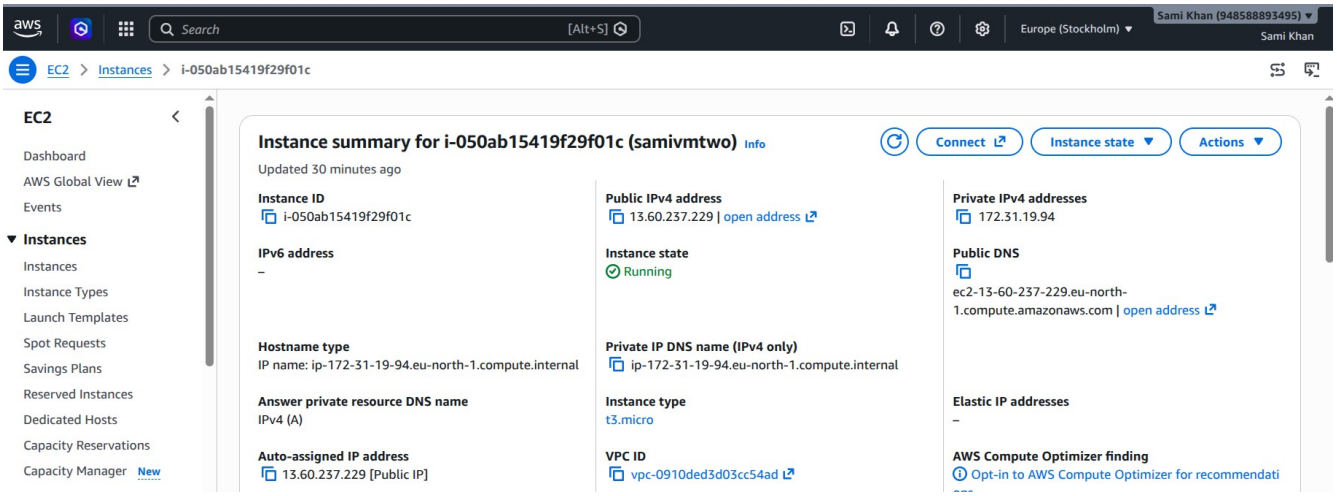
Initially files were not copied when I was opening public ip in browser it was showing nginx default page then default files were removed and actual repo files were copied to nginx.

```
ubuntu@ip-172-31-19-94: ~/Websitedeployment — ssh -...
~/Documents/awsnewkey
ubuntu@ip-172-31-19-94: ~/Websitedeploy ×   sami@sami: ~/Documents/github — /home/sar

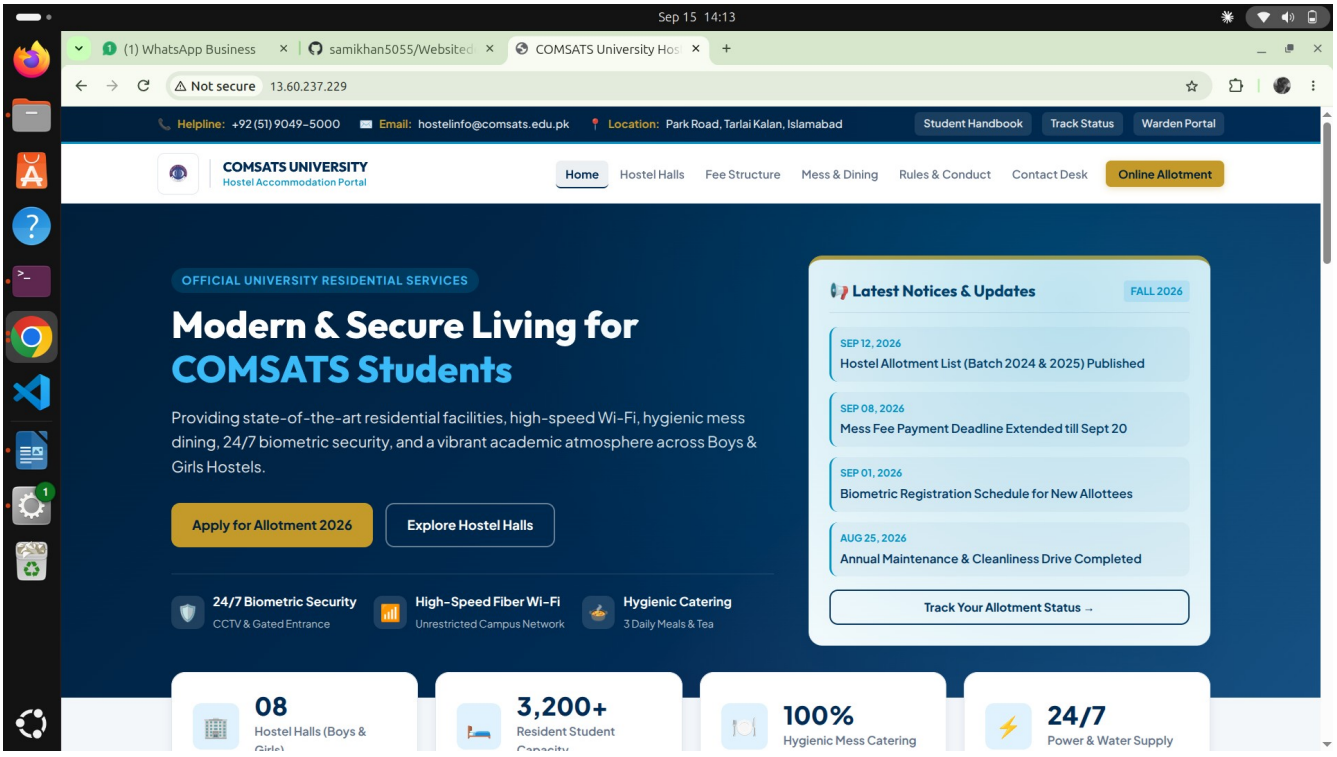
Sep 15 08:57:49 ip-172-31-19-94 systemd[1]: Starting nginx.service - A high perfo>
Sep 15 08:57:49 ip-172-31-19-94 systemd[1]: Started nginx.service - A high perfor>
ubuntu@ip-172-31-19-94:~/Websitedeployment$ ls -la /var/www/html/
total 12
drwxr-xr-x 2 root root 4096 Sep 15 08:49 .
drwxr-xr-x 3 root root 4096 Sep 15 08:49 ..
-rw-r--r-- 1 root root  615 Sep 15 08:49 index.nginx-debian.html
ubuntu@ip-172-31-19-94:~/Websitedeployment$ sudo rm -rf /var/www/html/*
ubuntu@ip-172-31-19-94:~/Websitedeployment$ ls
apply.html    fee-structure.html  index.html          rules-regulations.html
contact.html  hostels.html        mess-catering.html  style.css
ubuntu@ip-172-31-19-94:~/Websitedeployment$ sudo cp -r . /var/www/html/
ubuntu@ip-172-31-19-94:~/Websitedeployment$ ls -la /var/www/html/
total 160
drwxr-xr-x 3 root root  4096 Sep 15 09:01 .
drwxr-xr-x 3 root root  4096 Sep 15 08:49 ..
drwxr-xr-x 7 root root  4096 Sep 15 09:01 .git
-rw-r--r-- 1 root root 14901 Sep 15 09:01 apply.html
-rw-r--r-- 1 root root 13743 Sep 15 09:01 contact.html
-rw-r--r-- 1 root root 15633 Sep 15 09:01 fee-structure.html
-rw-r--r-- 1 root root 21688 Sep 15 09:01 hostels.html
-rw-r--r-- 1 root root 20220 Sep 15 09:01 index.html
-rw-r--r-- 1 root root 18117 Sep 15 09:01 mess-catering.html
-rw-r--r-- 1 root root 11593 Sep 15 09:01 rules-regulations.html
```

7. Verifying the Deployed Website

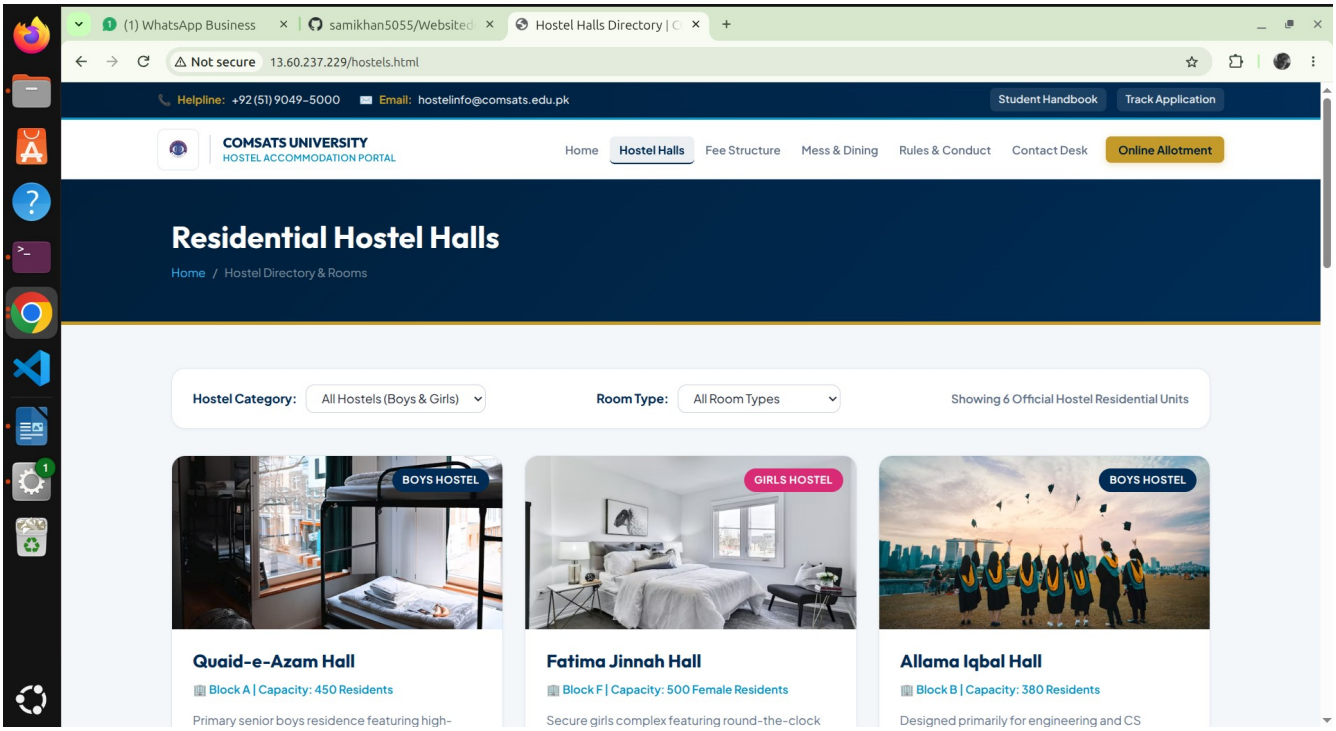
The EC2 instance is running and the console is showing the public IP from which we will open our site in the browser, which is **13.60.237.229**. This is the only address through which the deployed site can be reached. It is checked here because a stopped instance would make the site unreachable.



Now we have opened our deployed site in the browser using the link `http://13.60.237.229/`. The homepage loads correctly, which means Nginx is serving the files from `/var/www/html`. This confirms that the site is successfully deployed on Nginx and running in the browser.

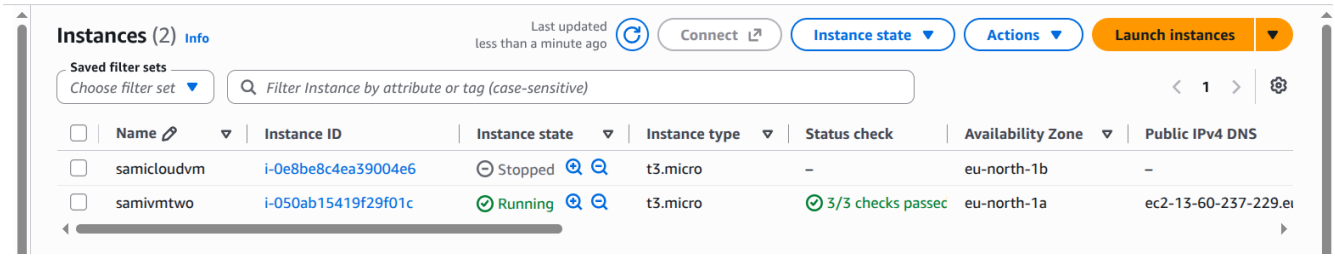


The hostel halls page loads with all of its styling and images intact. This is shown to confirm that the internal links and the CSS file are also being served properly and not only the homepage.



8. Final State of the EC2 Instances

And in the end this screenshot shows the current instance that is running and the other one that is closed. The running instance is the one serving the deployed website on `13.60.237.229`. It is included to record the final state of the resources at the end of the task.



Click on provided text to open the site; **COMSATS HOSTEL**